



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

Cost Manager RESTful Web Services

Final Project in
Asynchronous Server-Side
Development Course

PLEASE NOTE THAT THIS DOCUMENT MIGHT CHANGE DURING THE WEEKS TILL THE SEMESTER ENDS. IF CHANGES TAKE PLACE, THESE CHANGES WILL BE LISTED AT THE BOTTOM OF THIS DOCUMENT. CHANGES IN THIS DOCUMENT WILL BE INTRODUCED TO CLARIFY THE REQUIREMENTS ONLY. THE REQUIREMENTS WON'T BE CHANGED. **MAKE SURE YOUR FINAL PROJECT MEETS THE REQUIREMENTS LISTED IN THIS DOCUMENT BY THE END OF THIS SEMESTER TO ENSURE THAT YOU UNDERSTAND THE REQUIREMENTS.**

THE FASTEST WAY POSSIBLE FOR GETTING CLARIFICATIONS REGARDING THIS DOCUMENT IS TO WRITE EACH QUESTION AS A SEPARATE POST IN OUR COURSE'S FORUM.



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

Database

The database should be a MongoDB database (using the MongoDB Atlas service). The data should be organized in collections that, at a minimum, include the `users`, the `costs`, and the `logs` collections. The use of MongoDB should implement the computed design pattern (explained below).

The database should support organizing costs by category (the following categories must be supported: food, health, housing, sports, and education).

The `users` collection should hold documents that (at a minimum) include the following properties: `id`, `first_name`, `last_name`, and `birthday`. The `id` and the `_id` are two different properties. Don't mix them. When creating the schema for working with Mongoose, please use the following types. The type of `id` is Number. The type of `first_name` and `last_name` is String. The type of `birthday` is Date.

The `costs` collection should hold documents that (at a minimum) include the following properties: `description`, `category`, `userid`, and `sum`. When creating the schema for working with Mongoose, please use the following types. The type of `description` is String. The type of `category` is String. The type of `userid` is Number. The type of `sum` is Double.



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

cannot use TypeScript). The application should work as a RESTful Web Service(s), which includes the following endpoints:

Adding Cost Items

It should be possible to send an HTTP request (POST) to add a new cost item. The parameters that should be sent to the server (at a minimum) are:

`description, category, userid,`

and `sum`. The `sum` must be a positive number. The response

should be a JSON document that describes the new cost item that was added. The

names of the parameters and the names of the properties in the JSON document the reply includes should be the same names that are used as the names of the properties of the documents added to the

`costs` collection. If the date and time at which the cost item was created are not passed to the server, the server will use the date and time at which the request was received. If an error happens, then a JSON document that describes the error should be returned. If adding the cost item succeeds, then a JSON document that describes the added cost item should be returned.

_____/api/add

Getting Monthly Report

It should be possible to send an HTTP request (GET) that includes (as part of the query string) the ID of the specific user for whom we want to get a JSON document that describes all cost items in a specific



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

the specific month and year that were passed over. These costs should be grouped according to the categories. Each cost should be described using the sum, the day of the month, and the description of that cost. The names of the parameters and the names of the properties in the JSON document the reply includes should be the same names that are used as the names of the properties of the documents added to the `costs` collections. If an error happens, a JSON document describing the error should be returned.

_____/api/report

The code responsible for generating the monthly report should implement the Computed Design Pattern. When a report is requested for a month that has already passed, that report will be saved for further requests for getting it (The server side doesn't allow adding costs with dates that belong to the past).

The following JSON shows how the report should look. The JSON returned in your project should look the same. Please note that a category without any cost items should still be mentioned in the report (e.g. "health" and "housing" are shown in the sample below, even though they don't have costs).

{



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

```

        {"sum":14,"description":"baigale","day":22}
      ]
    },
    {
      "education": [
        {"sum":82,"description":"math
book","day":10},
        {"sum":112,"description":"java
book","day":12},
        {"sum":182,"description":"dictionary","day":22}
      ]
    },
    {
      "health": []
    },
    {
      "housing": []
    },
    {
      "sport": []
    }
  ]
}

```

Getting The Details of a Specific User

It should be possible to send an HTTP request (GET) to get the details of a specific user. The user ID will be sent as part of the URL address. The reply should be in JSON, and it will include the first name, the last name, the id, and the total costs of that specific user. The names of the properties of that reply should be `first name`, `last name`, `id`, and `total`. The following URL is a sample for getting the details of the user whose ID is 123123. If an error happens, then a JSON document that describes the error should be returned.

_____ /api/users/123123

Developers Team

It should be possible to send an



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

document should be the same names that were used in the `users` collection. These names shouldn't be stored in the database. That would be a problem given the requirement for submitting the project with an empty database, except for a single imaginary user. These names can be either hardcoded (be part of the code) or stored in the `.env` file. The JSON document should include the first + last names of each team's members. The JSON document shouldn't include any additional data. Just the first name and the last name. If an error happens, a JSON document describing the error should be returned.

_____/api/about

List of Users

It should be possible to send an HTTP request (GET) to get a JSON document that describes all users. The names of properties in the JSON document the reply includes should be the same names that are being used as the names of the properties of the documents added to the `users` collection. If an error happens, a JSON document describing the error should be returned.

_____/api/users

List of Logs

It should be possible to send an HTTP request (GET) to get a JSON document that describes



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

added to the `logs` collection. If an error happens, a JSON document describing the error should be returned.

_____ /api/logs

Adding User

It should be possible to send an HTTP request (POST) to add a new user. The parameters that should be sent to the server (at the minimum) are: `id`, `first_name`, `last_name`, and `birthday`. The response should be a JSON document that describes the new user that was added. The names of the parameters and the names of the properties in the JSON document the reply includes should be the same names that are used as the names of the properties of the documents added to the `users` collection. If an error happens, then a JSON document that describes the error should be returned. If adding the user succeeds, then a JSON document that describes the added user should be returned. The database cannot have more than one document describing the same user. If there is a try to add a user that the `users` collection already includes a document that describes that specific user then an error will take place and a JSON document describing that error will be returned back to the client.

_____ /api/add

Error Message

When a JSON document that



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

You should verify that the RESTful Web Services work as expected by running the test program that the final project document includes.

Make sure the code that works with MongoDB is placed inside a separate folder, whose name is `models` (as learned in class).

Log Messages

You should use the Pino library (<https://getpino.io>) for creating log messages that will be saved to the MongoDB database. Log message should be written to the database for every HTTP request the server-side receives and, in addition, whenever an endpoint is accessed.

The .env File

The project should include the use of the `.env` file (as explained in class).

Four Processes

The project should include the development of four processes (micro services). The first will handle the admin stuff (Getting Logs). The second will handle all user-related tasks (Getting Details of Specific User, Adding User, List of Users). The third will handle all cost-related operations (Adding Cost Item, Getting Monthly Report). The fourth will handle any admin-related operations (Developers Team), such as getting the details of the developers who developed the application. There are more than a few possibilities to develop



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

you will need to fill in the details on a form (check submission guidelines item 6). That form will require you to provide four different URL addresses. Each address would be for a specific process (micro service). If the four processes are deployed on the same server, then they must have different ports. You cannot have several processes on the same computer using the same port. If the four processes (micro services) are deployed on separate servers, then they might use the same port. That shouldn't be a problem. Please note that developing one project with four routes (each route handling a separate collection of endpoints) won't be a fulfillment of the requirement for separate processes.

Unit Tests

You should develop detailed unit tests for each one of the endpoints. You can choose which programming language and which libraries to use when coding these unit tests.

Code Style

The code in JavaScript should follow the guidelines listed in the Professional JavaScript Guide book, which you can find at https://www.amazon.com/Professional-JavaScript-Style-Guide-Maintainable-ebook/dp/B0GK11M6NW/ref=sr_1_2. Make sure that you don't forget to add comments. Please follow the course message board where you can get this book for free.



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

You should deploy your final project on a server (or servers) connected to the web. Each microservice should be implemented in a separate process. Don't forget to fill in the form (listed as one of the submission guidelines) with the URL addresses of the four microservices.

Submission Deadline

The deadline for submitting the final project will be published in our course message board.

Submission Guidelines

Here are the guidelines for submitting this project. You should carefully follow these guidelines. If a question arises, you should post it to the course forum in order to get a detailed, accurate answer. Points will be deducted when the submission doesn't meet the following guidelines (e.g., if the submitted PDF is not properly organized in order to allow code review, points will be deducted).

1. You should create a short video (try to make it up to 60 seconds) that shows how the project runs. You should upload that video to YouTube and make sure you upload it as an **unlisted** video.
2. You should pack the entire project (including the testing code



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

Otherwise, the ZIP file will be too big to upload to Moodle. You should upload these two files only. Please make sure to upload them separately. Don't pack them in a single ZIP file!

3. You should create a PDF file, and its name should include the first name and the last name of the team manager with `\_` between (e.g., moshe_israeli.pdf). The name should include lowercase letters only.

4. The PDF file should include (at the beginning of it) the following:

- The first and last names of the development team manager.
- First name + Last name + ID + Mobile Number + Email Address of each one of the team members.
- Link to the video you created (item 1). This link should be clickable.
- In addition, the PDF file can include any additional guidelines that will usually be added to the readme.txt file.
- Summary of the use you made in two collaborative tools (at a minimum). The summary should be up to 100 words.
- The PDF file should include the source code of all the files you wrote. The PDF file should include the name of each file next to its code. Make sure that lines are not broken. Make sure this PDF file is properly organized in order to allow the code review. Make sure you include in this PDF file all the code you wrote in JavaScript. The code in each file should be preceded by the name of that file.

5. The team manager should submit the PDF file and the ZIP file in the assignment box that will



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

the time on the server (on which Moodle is running) and the time on your end. Due to this difference, you should treat the deadline published on our Moodle website as if it were 30 minutes earlier. Late submissions won't be accepted. Submissions of projects developed by a single student won't be accepted. Submissions of projects developed by students from different groups won't be accepted. It is not possible to get a delay. Teams with justified reasons for getting a delay (in accordance with the college guidelines) will be handled separately. They won't submit through the assignment box of this project, and they won't get their grade along with all others.

6. Please fill out the form at _____ with the URL addresses that we can use to test the project you have developed and deployed online.

7. When submitting the project, the database should be empty, except for having a single imaginary user (document in users) with the following details:

id: 123123

first_name: mosh

last_name: israeli

8. The project will be carried out in teams of two students and will also include a 10% teamwork assessment using the tools presented during the course. A student who submits the project alone will lose 10% of the project grade. Students who submit alone will automatically lose 10% from the project grade. When submitting a project, it is not possible to get a delay. Students who don't submit on



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

Code for Testing The Project

THE FOLLOWING CODE IS JUST A CODE SAMPLE THAT CAN TEST THE FINAL PROJECT. DURING THE TEST THAT WILL TAKE PLACE WHEN GRADING THE FINAL PROJECT, WE WILL USE A DIFFERENT CODE (SIMILAR TO THIS ONE). **YOU ARE EXPECTED TO TEST YOUR FINAL PROJECT (BEFORE YOU SUBMIT IT) USING THE FOLLOWING CODE (DON'T FORGET TO COMPLETE THE MISSING URL ADDRESSES THAT SHOULD BE ASSIGNED TO VARIABLES a,b,c, and d (AS WAS EXPLAINED IN CLASS)).**

```
import requests
Import sys

filename =
input("filename=")

#The first will handle the
logs. (a)
#The second will handle
all user-related tasks.
(b)
#The third will handle all
cost-related tasks. (c)
#The fourth will handle
any admin-related tasks
```



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

```
"
c =
"
d =
"

output =
open(filename, "w")
sys.stdout = output

print("a="+a)
print("b="+b)
print("c="+c)
print("d="+d)

print()

print("testing getting the
about")
print("-----
-----")

try:
    text = ""

    #getting details of team
    manager

    url = d + "/api/about/"

    data = requests.get(url)

    print("url="+url)

    print("data.status_code="+str(data.status_code))

    print(data.content)

    print("data.text="+data.text)

    print(data.json())

except Exception as e:

    print("problem")

    print(e)

print("")

print()
```



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

```
text = ""

#getting the report

url = c + "/api/report/?
id=123123&year=2026&month=1"

data = requests.get(url)

print("url="+url)

print("data.status_code="+str(data.status_code))

print(data.content)

print("data.text="+data.text)

print(text)

except Exception as e:

    print("problem")

    print(e)

print("")

print()

print("testing adding cost
item")

print("-----
-----")

try:

    text = ""

    url = c + "/api/add/"

    data = requests.post(url,
        json=
{'userid':123123,
'description':'milk
9','category':'food','sum':8})

    print("url="+url)

    print("data.status_code="+str(data.status_code))

    print(data.content)
```



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

```
print()

print("testing getting the
report - 2")

print("-----
-----")

try:

    text = ""

    #getting the report

    url = c + "/api/report/?
id=123123&year=2026&month=5"

    data = requests.get(url)

    print("url="+url)

    print("data.status_code="+str(data.status_code))

    print(data.content)

    print("data.text="+data.text)

    print(text)

except Exception as e:

    print("problem")

    print(e)

print("")
```

Questions and Answers

.1

האם יש לבצע בדיקות validation
לכל data אשר מגיע ל-end-
?point
כן. <=



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

להקפיד עליה (אחרת יורדות נקודות) היא ליצור תיקיה בשם models שבתוכה מופיעים קבצים עם קוד ב-JavaScript אשר כל אחד מהם כולל קוד שמשתמש ב-mongoose וש אחראי ליצירת ה-schema ולמיפוי בין constructor function ו-collection מסויים ב-MongoDB.

3.
האם השמות של הקבצים בתיקיה models חייבים לכלול אותיות קטנות בלבד? <= על פי ההנחיות ב-style guide התשובה היא כן. יחד עם זאת, כיוון שמאד מקובל לתת לקבצים בתוך התיקיה models שמות אשר מתחילים באות גדולה, לא ירדו נקודות למי שיעשה כך.

4.
האם חייבים להוסיף לקוד comments בסגנון JSDoc? <= לא. אין דרישה במסמך של הפרוייקט לעשות זאת. יש שני סוגים של הערות שחשוב להוסיף. הערה בסגנון C אשר מתפרשת על פני יותר משורה אחת, והערה בסגנון ++C אשר מתפרשת על פני שורה אחת ומופיעה מעל לשורה הקוד שמוסברת (או שורות הקוד שמוסברות). בתהליך הבדיקה, במידה שיש כ-8-9 שורות (לפחות) ללא הערה בסגנון ++C אז נקודות ירדו על מחסור בהערות. לא לשכוח, שבכל הנוגע להערות בסגנון C כיוון שאתם מממשים את Computed חשוב להוסיף למקום המתאים הערה



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

המילה model כשהיא מופרדת
בנקודה לפני ונקודה אחרי מייד
לפני הסיומת של הקובץ (js)?
=< כן (למרות שה-style guide
שלנו מכתוב אחרת, לא ירדו
נקודות כיוון שזה מאד מקובל).

6.
האם ניתן לכלול בשמות הקבצים
שכוללים את הקוד שמבצע unit
testing את המילה test בצירוף
נקודה מפרידה?
=< כן (למרות שה-style guide
שלנו מכתוב אחרת, לא ירדו
נקודות כיוון שזה מאד מקובל).

7.
האם המערכת אמורה לתמוך
בקבלת דו"ח חודשי לחודש
הנוכחי? לחודשים עתידיים?
=< כן. במסמך של הפרוייקט לא
מופיע שום דרישה או אפיון
שאפשר להבין מהם אחרת. אין
במסמך של הפרוייקט הנחיה לכך
שהפרוייקט יתמוך בקבלת דו"חות
לחודשים ששייכים לעבר בלבד.

8.
האם הודעות ה-log שנוצרות
יכולות לכלול גם הודעות log
נוספות לאלה שנדרש?
=< כן.

9.
האם ניתן לפתח ארבעה
פרוייקטים נפרדים ולעשות לכל
אחד מהם deployment בנפרד
ב-web?
=< כן.

10.
האם קיימת דרישה לפתח ארבעה
פרוייקטים נפרדים על מנת לקבל
תוצאה סופית של 4 processes



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

האם לבצע בדיקה האם מדובר ב-
user שאכן קיים?
=< כן.

12.
האם הבדיקה של ה-id של user על
מנת לוודא שמדובר ב-user קיים
צריך שתתבצע מול end point
חדש אשר יפותח לצורך זה?
=< בהחלט אפשרי. במקרה
שמוסיפים end point חדש (או
חדשים) יש להוסיף אותו (אותם)
למקבץ ה-end points המתאים.
כך, למשל במקרה של הוספת
end point לבדיקת קיומו של
user id מסויים אשר נוסף לשאר
ה-end points בנוגע ל-users כך
שהוא יפעל באותו process אשר
אחראי לכל הפעולות האחרות
בנוגע ל-users

13.
האם ניתן לפתח end points
נוספים מעבר לאלה שנדרש
לפתח על פי המסמך של
הפרוייקט?
=< כן.

14.
האם פיתוח פרוייקט אחד שכולל
הגדרה של 4 routes באופן שבו
כל הדרישות של הפרוייקט ימומשו
באמצעות routes אלה... ייחש
לפרוייקט אשר פותח על פי
הדרישות במסמך זה?
=< לא.

15.
כיצד אמור להיראות report
כאשר אין בכלל הוצאות להציג?
=< ה-report אמור להיראות
כדלקמן:



nodejs_final_project_202607

מעודכן אוטומטית כל 5 דקות

```
    {
      "food": []
    },
    {
      "education": []
    },
    {
      "health": []
    },
    {
      "housing": []
    },
    {
      "Sport": []
    }
  ]
}
```

Changes in Document

1.

August 21, 2026

The "sport" category should be using lowercase only. Typo was fixed in the code sample for unit testing.

2.

August 29, 2026

The text "The sum must be a positive number." was added to the requirements for adding a new cost.

3.

August 29, 2026

The "Make sure the comments you write in your code are in English." text was added to the section that refers that coding style guidelines.